
ATTENTION IS ALL YOU NEED... BUT WHERE?

A CONTROLLED STUDY OF ATTENTION PLACEMENT IN CNN-LSTM ARCHITECTURES

Samih Mejhoudi
School of Computer Science and
Technology
Harbin Institute of Technology, Shenzhen
Advanced Artificial Intelligence Course
Lab
25SF51011@stu.hit.edu.cn

Kuo-Kun Tseng
School of Computer Science and
Technology
Harbin Institute of Technology, Shenzhen
kktseng@hit.edu.cn

July 2026

ABSTRACT

CNNs and LSTMs remain widely used for sequence classification, and attention mechanisms are now commonly added to these architectures. However, the position of the attention layer is often chosen by convention rather than being studied as a design decision. This paper investigates whether attention placement affects performance and whether the effect depends on sequence length. A shared CNN-LSTM backbone is used, with an identical attention module inserted at three positions: before the CNN, between the CNN and the LSTM, and after the LSTM. All other architectural components, hyperparameters, and random seeds are held fixed across variants, allowing a controlled comparison of attention placement. We evaluate all three placements on three text classification datasets spanning short (AG News), medium (IMDB), and long (ArXiv) sequences, using five random seeds per configuration. On short sequences, placement has only a modest effect, with post-LSTM attention outperforming earlier placements by approximately one percentage point. On medium and long sequences, the gap widens sharply: post-LSTM attention remains stable and accurate (85% on IMDB, 84% on ArXiv), while earlier placements become increasingly unstable and

exhibit substantial performance degradation on long sequences. A follow-up convergence analysis shows that earlier placements are not incapable of learning; rather, they exhibit substantially slower optimization dynamics and fail to match the performance of post-LSTM attention even with extended training. These findings suggest that attention placement should be treated as a first-class architectural design choice, particularly for medium- and long-sequence classification tasks.

Keywords: Structural Intelligence, Convolutional Neural Networks, Recurrent Neural Networks, Attention Mechanism, Sequence Length Analysis

1 Introduction

CNNs and LSTMs have been standard components for sequence classification for years [1]. Attention layers are now routinely added on top of them to improve performance [2, 3]. In most published work, though, the attention layer sits at a single, fixed point in the architecture, chosen by convention rather than tested against alternatives [5, 6, 7]. Whether that placement actually matters, and whether its effect depends on sequence length, has received relatively little direct study.

This issue has practical importance. CNN–LSTM–attention hybrids are still widely used, especially in applied domains like intrusion detection, sentiment analysis, and document classification [8, 9, 11], where a full Transformer is often too expensive to train or deploy. If placement affects performance meaningfully, then every time someone picks a spot for attention without testing it, they are making an unexamined design choice.

This paper addresses three research questions. Does attention placement affect classification accuracy at all? If it does, does the size of that effect grow or shrink with sequence length? And if performance differences show up, are they just a matter of slower learning, or do they persist even when given much more training time?

To answer these questions, we build one CNN–LSTM backbone and insert the same attention module at three positions: before the CNN, between the CNN and the LSTM, and after the LSTM. Everything else, including the architecture, hyperparameters, and random seeds, remains fixed across the three variants. We test all three on three datasets chosen specifically for their different sequence lengths: AG News (short), IMDB (medium), and ArXiv abstracts (long), with five random seeds per configuration. This design follows the same length-aware, controlled-comparison logic used by Hallam and Tseng [16] to study positional-encoding fusion in Transformers, applied here to a different architecture and a different design variable.

On short sequences, placement barely matters. On medium and long sequences, a substantially different pattern emerges. Post-LSTM attention stays stable and accurate across all three datasets.

Attention placed earlier becomes increasingly unstable and, on the longest sequences, performs close to random guessing under standard training. This pattern echoes prior findings that attention-placement ablations in other domains, such as face manipulation detection [12] and crack detection [13], also show sensitivity to where attention sits in a network. A follow-up experiment with extended training shows these early placements can learn, just much more slowly, and they still do not catch up to post-LSTM attention even after several times more epochs.

This paper makes three main contributions: a controlled comparison of three attention placements inside one fixed CNN–LSTM architecture across three sequence-length regimes; evidence, backed by paired-seed statistical testing, that the gap between placements widens as sequences get longer; and a convergence analysis showing that early-placement instability is partly, but not fully, a matter of slow optimization rather than an inability to learn at all.

The rest of the paper is organized as follows. Section 2 covers related work on CNN–LSTM–attention hybrids and prior placement studies. Section 3 describes the architecture, attention module, datasets, and training setup. Section 4 presents results across all three datasets. Section 5 discusses sequence-length effects, training efficiency, convergence behavior, and the limitations of this experimental design. Section 6 concludes.

2 Related Work

2.1 CNN-Based Text Classification

Convolutional networks were adapted for text classification by treating a sentence as a one-dimensional sequence and sliding small filters across it to detect local patterns, similar to how n-gram features work but learned rather than hand-designed. A filter of width three, for instance, picks up on short local phrases regardless of where they appear in the sequence. This made CNNs fast to train and effective at picking up local cues such as short phrases or keyword combinations, and they remain a common first layer in hybrid text classification systems, including all three of the model variants studied in this paper, precisely because they extract useful local structure before any sequential modeling happens.

2.2 LSTM-Based Text Classification

Where CNNs are good at local patterns, they have no real mechanism for tracking dependencies that span the whole sequence. Long Short-Term Memory networks were designed to address this, using gated units that let information persist across many time steps without the vanishing-gradient problems that plague plain recurrent networks [1]. Encoder–decoder architectures built on this idea became a standard way to represent variable-length sequences as fixed-size vectors for downstream

tasks [4]. For text classification specifically, an LSTM reading a sequence left to right, or in both directions, gives a model some sense of order and long-range context that a CNN alone cannot provide.

2.3 CNN–LSTM Hybrid Architectures

Combining the two is a natural next step, and the combination shows up frequently in applied work: a CNN extracts local features first, then an LSTM models how those features relate across the sequence. This pattern appears across a range of domains, including network intrusion detection [5, 8, 9, 11], plant disease classification from text-adjacent sensor data [6], imbalanced cyber-physical system monitoring [10], and web content classification [7]. In nearly all of these, attention is added as a further enhancement on top of the CNN–LSTM base, typically placed at whatever point the authors found convenient, and reported as part of an ablation table showing that the full model beats a version without attention. Placement itself is rarely questioned.

2.4 Attention Mechanisms in Sequence Modeling

Attention was introduced as a way to let a model weight different parts of an input differently, originally to fix a bottleneck in encoder–decoder translation, where a single fixed vector had to represent an entire source sentence [2]. This idea was later generalized and scaled up into the Transformer architecture, which dispensed with recurrence entirely and used attention as the primary mechanism for modeling sequence relationships [3], with follow-up work refining the internal components of that architecture [18]. Outside of pure Transformers, attention has also been studied as an add-on module whose position within a network affects results: ablations in face manipulation detection [12], crack detection [13], respiratory sound classification [14], and DNA sequence modeling [15] all report that where attention sits changes performance, though each of these findings is local to one task and one architecture, not the subject of a dedicated study.

2.5 Research Gap

Taken together, two things are missing from this body of work. First, no CNN-LSTM-attention study we found treats placement as the primary variable under test; it is always a secondary detail attached to a larger system. Second, none of the placement-sensitive findings outside the CNN-LSTM setting are examined across different sequence lengths, so it remains unclear whether placement effects are a fixed property of an architecture or something that depends on how long the input is. The closest methodological precedent for the second point is Hallam and Tseng [16], who show that a different design choice, how positional encodings are fused with embeddings in

a Transformer, has almost no effect on short sequences but a substantial one on long documents. Their length-aware, controlled-comparison approach is the model this paper follows, applied to a different architecture and a different design choice: not how information is fused, but where an attention module is placed.

3 Methodology

3.1 Architecture Overview

All three model variants in this study share a single backbone, differing only in where a single attention module is inserted. An input sequence of token IDs is first mapped to dense vectors by an embedding layer, producing a sequence of embeddings $E \in \mathbb{R}^{T \times d}$ for a sequence of length T . This is passed through a one-dimensional convolution with ReLU activation, extracting local n-gram-style features from the embedded sequence. The resulting feature sequence is processed by a single-layer LSTM, which models dependencies across the full sequence. A final linear layer maps the resulting representation to class logits. Without attention, this reduces to a standard CNN–LSTM text classifier of the kind used across the applications reviewed in Section 2.3. Exact layer sizes and hyperparameters are given in Section 3.4 and Table 2.

3.2 Attention Mechanism

We use a small additive attention module, following the alignment mechanism introduced by Bahdanau et al. [2]. Given a sequence of vectors $x_1, \dots, x_T$ at whichever stage attention is inserted, the module first computes a scalar score for each position:

$$e_i = W_a x_i + b, \quad (1)$$

where W_a and b are learned parameters. These scores are normalized into attention weights with a softmax over the sequence dimension:

$$\alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^T \exp(e_j)}. \quad (2)$$

The same module is inserted at three different points in the backbone, and how its output is used depends on where it sits. We treat this distinction as a property of the design space rather than an implementation detail to gloss over: the three placements below use the same attention module, but its role necessarily changes depending on where it sits in the network.

3.3 Attention Placement Variants

We evaluate three placements of the attention module within the shared backbone, illustrated in Figure 1.

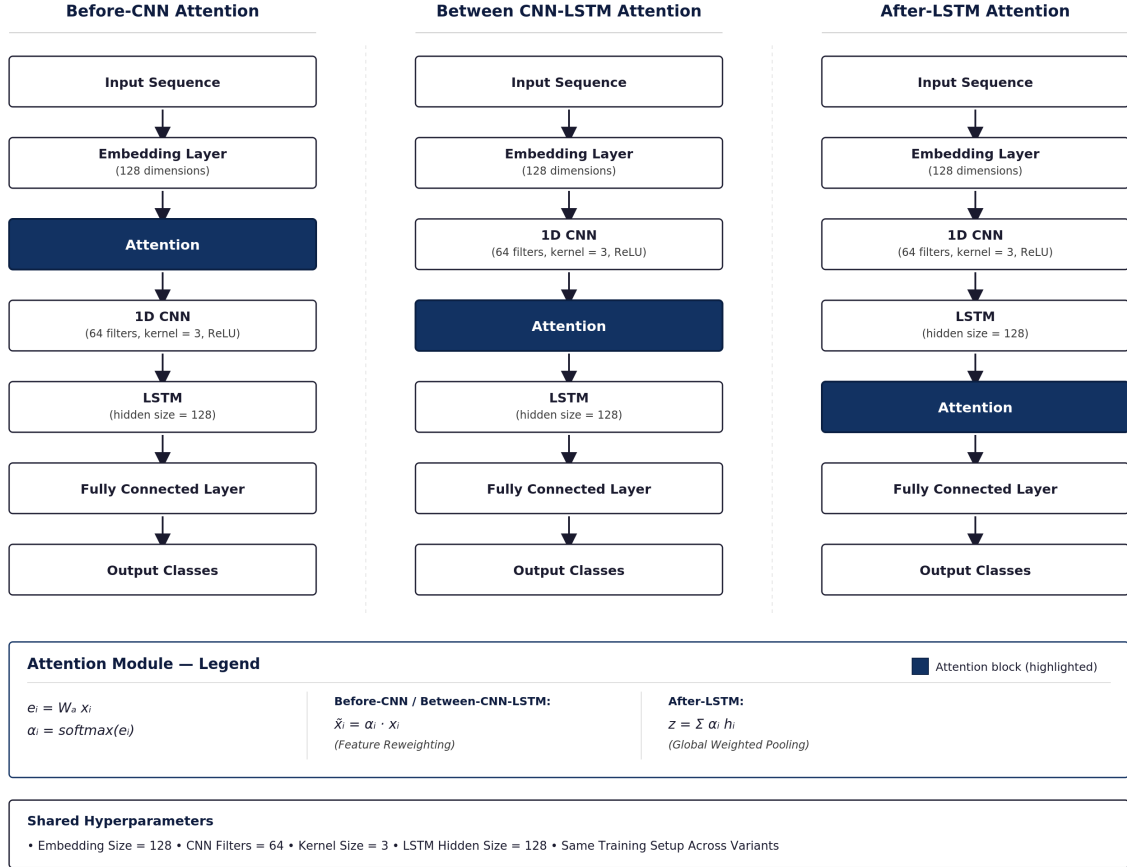


Figure 1: Attention placement variants in the CNN–LSTM architecture: (a) before the CNN, (b) between the CNN and the LSTM, and (c) after the LSTM. Early placements reweight the sequence elementwise (Equation 3); the post-LSTM placement pools it into a single vector (Equation 4).

3.3.1 Before-CNN Attention

Attention operates directly on the embedded sequence E , before any convolution or recurrence. The weights from Equation 2 are used to reweight the sequence elementwise, preserving sequence length so that the CNN still receives a full sequence as input:

$$\tilde{x}_i = \alpha_i x_i. \quad (3)$$

The reweighted sequence is then passed into the CNN as usual.

3.3.2 Between CNN and LSTM Attention

Attention operates on the CNN’s output features, after local patterns have already been extracted but before the LSTM processes them sequentially. Equation 3 applies again here, this time reweighting feature vectors rather than raw embeddings, and the sequence is passed on to the LSTM.

3.3.3 After-LSTM Attention

Attention operates on the LSTM’s hidden states $h_1, \dots, h_T$. Since nothing downstream needs sequence structure anymore, the weights are instead used to collapse the full sequence into a single pooled vector:

$$z = \sum_{i=1}^T \alpha_i h_i. \quad (4)$$

This is the only placement in which attention discards sequence structure entirely, since the classifier that follows expects a single fixed-size vector regardless.

3.4 Datasets

We evaluate all three placements on three text classification datasets, chosen specifically because they differ in typical sequence length, allowing us to test whether placement effects change with input length: AG News, a four-class news topic classification dataset with short sequences; IMDB, a binary sentiment classification dataset of movie reviews with medium-length sequences; and ArXiv, an eleven-class scientific abstract classification dataset with long sequences. Table 1 summarizes the size, class count, and maximum sequence length used for each.

Table 1: Dataset statistics

Dataset	Train size	Test size	Classes	Max length
AG News	120,000	7,600	4	40
IMDB	25,000	25,000	2	200
ArXiv	28,388	2,500	11	1000

3.5 Training Setup

AG News and IMDB were trained locally, while the longer ArXiv dataset was trained using an NVIDIA T4 GPU on Kaggle due to its substantially higher computational requirements. All three placement variants use identical hyperparameters, listed in Table 2, and are optimized with Adam by minimizing cross-entropy loss:

$$\mathcal{L} = - \sum_{c=1}^C y_c \log(\hat{y}_c). \quad (5)$$

Each variant is trained with five random seeds per dataset, and the same seed is used across all three placements within a matched run, so any accuracy difference under a given seed reflects placement rather than differences in weight initialization or data ordering. For each dataset, all three placements are trained across all five seeds, giving 15 runs per dataset and 45 runs total across the three main datasets. Vocabulary is built independently per dataset from its training split and capped at 20,000 tokens.

Table 2: Training configuration

Setting	Value
Embedding dimension	128
CNN channels / kernel size	64 / 3
LSTM hidden size	128
Optimizer	Adam
Learning rate	1×10^{-3}
Batch size	32
Epochs	3
Seeds per configuration	5 (seeds 1–5)
Total training runs	45

Figure 2 confirms that this shared configuration produces near-identical parameter counts across all three placements, differing by fewer than 100 parameters out of roughly 2.68 million. Consequently, any performance differences reported in Section 4 cannot be attributed to differences in model capacity.

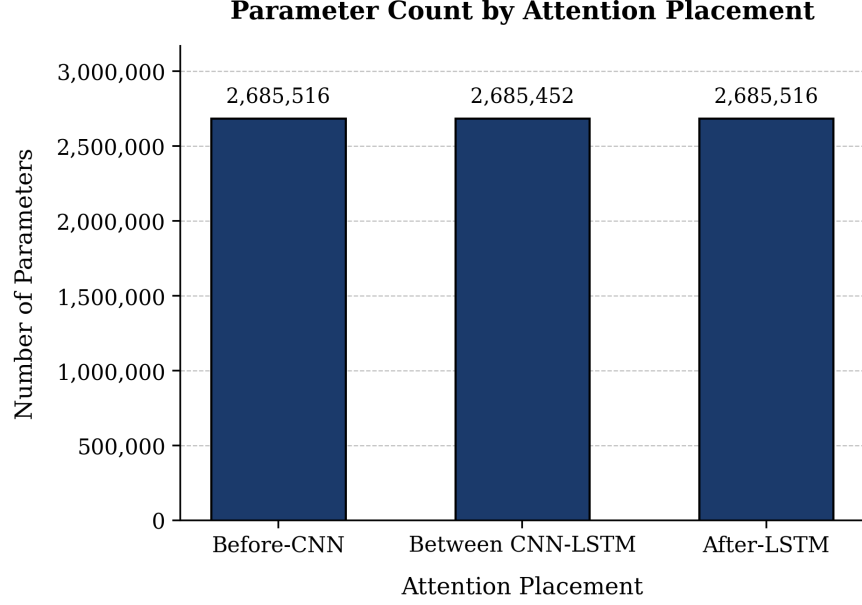


Figure 2: Parameter count by attention position. Differences across the three placements are negligible, confirming a fair comparison.

3.6 Evaluation Metrics

Following standard text-classification practice, model performance is evaluated using accuracy, macro-averaged precision, macro-averaged recall, and macro-averaged F1-score.

$$\text{Accuracy} = \frac{\text{Correct predictions}}{\text{Total predictions}}. \quad (6)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}. \quad (7)$$

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}. \quad (8)$$

To determine whether observed performance differences are statistically meaningful rather than the result of random variation across seeds, a paired t -test is performed on the five per-seed accuracy values for each pair of attention placements. Because the same seeds are used across all placements, the comparison is performed on matched runs:

$$t = \frac{\bar{d}}{s_d / \sqrt{n}}, \quad (9)$$

where $\bar{d}$ is the mean difference in accuracy between paired runs, s_d is the standard deviation of those differences, and $n = 5$ is the number of matched random seeds.

4 Experiments and Results

4.1 AG News Results

Table 3: Performance metrics on AG News. Values are reported as mean $\pm$ standard deviation across five random seeds.

Position	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
Before CNN	90.38 $\pm$ 0.31	90.46 $\pm$ 0.28	90.38 $\pm$ 0.31	90.39 $\pm$ 0.30
Between	90.42 $\pm$ 0.41	90.53 $\pm$ 0.33	90.42 $\pm$ 0.41	90.40 $\pm$ 0.39
After LSTM	91.32 $\pm$ 0.17	91.36 $\pm$ 0.15	91.32 $\pm$ 0.17	91.32 $\pm$ 0.16

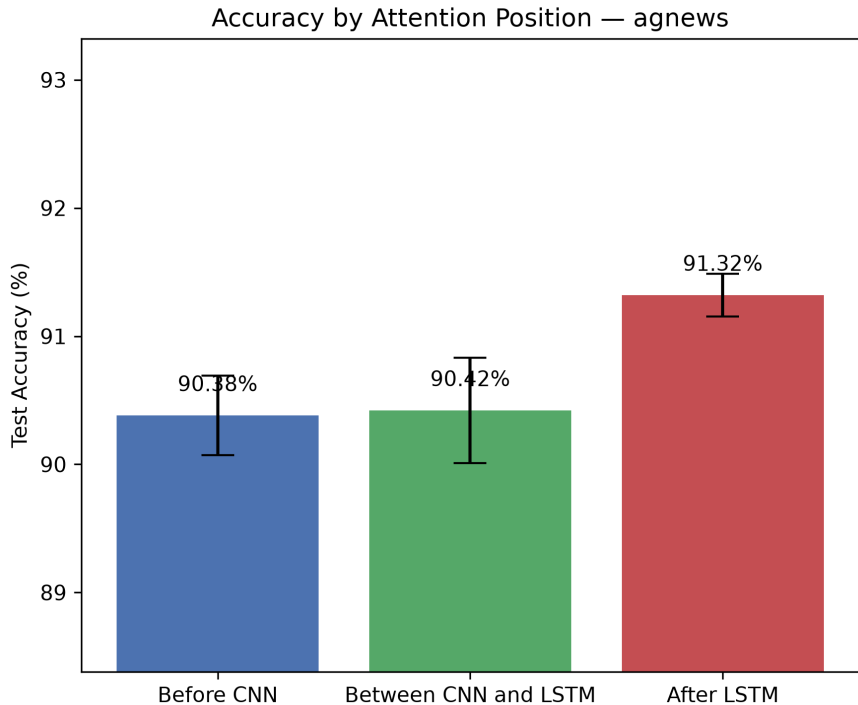


Figure 3: Test accuracy by attention placement on AG News, with error bars showing standard deviation across five seeds.

Results Discussion. On AG News, the shortest-sequence dataset considered in this study, all three attention placements achieve similar performance, with less than one percentage point separating the best and worst configurations. Post-LSTM attention achieves the highest accuracy at 91.32%, compared with 90.38% for Before-CNN attention and 90.42% for Between attention. The Before-CNN and Between variants perform nearly identically, differing by only 0.04 percentage points.

Overall, attention placement has only a modest effect on short-sequence classification, although the post-LSTM variant consistently produces the strongest results across all reported metrics. The relatively small gap observed here contrasts with the substantially larger differences reported for the longer-sequence datasets in Sections 4.2 and 4.3.

Statistical Significance Analysis. To determine whether the observed differences are statistically meaningful, a paired t -test was performed on the five per-seed accuracy values for each pair of placements. No significant difference was found between Before-CNN and Between attention ($t = -0.174$, $p = 0.870$), indicating that these two variants perform equivalently on AG News. In contrast, After-LSTM attention significantly outperformed both Before-CNN ($t = -8.198$, $p = 0.0012$) and Between ($t = -3.480$, $p = 0.0253$). Therefore, even on short sequences where the raw accuracy gap appears small, the advantage of post-LSTM attention is statistically significant and cannot be attributed to random variation.

4.2 IMDB Results

Table 4: Performance metrics on IMDB. Values are reported as mean $\pm$ standard deviation across five random seeds.

Position	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
Before CNN	53.57 $\pm$ 4.35	55.46 $\pm$ 3.68	53.57 $\pm$ 4.35	46.61 $\pm$ 8.14
Between	65.18 $\pm$ 12.27	65.34 $\pm$ 12.14	65.18 $\pm$ 12.27	60.01 $\pm$ 18.62
After LSTM	84.93 $\pm$ 0.18	85.03 $\pm$ 0.19	84.93 $\pm$ 0.18	84.92 $\pm$ 0.18

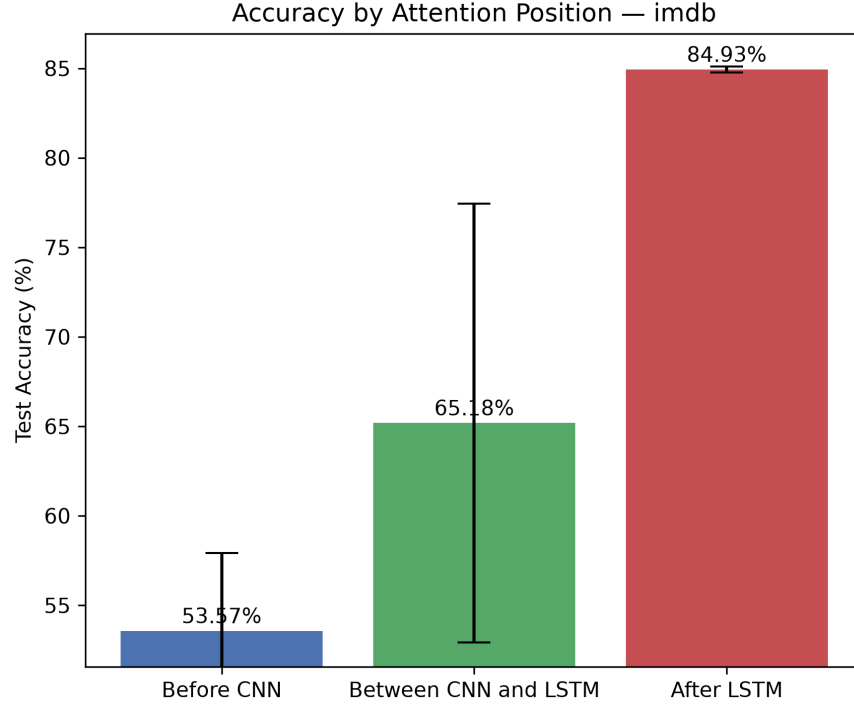


Figure 4: Test accuracy by attention placement on IMDB, with error bars showing standard deviation across five seeds.

Results Discussion. On IMDB, the medium-length dataset in this study, the small differences observed on AG News become large ones. After-LSTM attention reaches 84.93% accuracy with a standard deviation of only 0.18, essentially identical performance across every seed. Before-CNN and Between attention, by contrast, show both lower accuracy and far greater variance: 53.57% $\pm$ 4.35 and 65.18% $\pm$ 12.27, respectively. The large standard deviations indicate that these two placements do not learn reliably. Individual seeds for Before-CNN dropped as low as 50.7%, close to chance level for this binary classification task, while other seeds reached above 62%. Between shows the same pattern in a more extreme form, with some seeds near 50% and others above 76%. After-LSTM attention shows no such instability at any seed.

Statistical Significance Analysis. A paired t -test confirms that After-LSTM significantly outperforms both earlier placements (Before-CNN: $t = -15.002$, $p = 0.0001$; Between: $t = -3.233$, $p = 0.0319$). The difference between Before-CNN and Between is not statistically significant ($t = -2.051$, $p = 0.110$), despite their different mean accuracies, a consequence of the large variance in both. On IMDB, attention placement is no longer a matter of a small accuracy edge, as it was on AG News, but a matter of whether the model reliably learns at all.

4.3 ArXiv Results

Table 5: Performance metrics on ArXiv. Values are reported as mean $\pm$ standard deviation across five random seeds.

Position	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)
Before CNN	24.05 $\pm$ 8.88	16.81 $\pm$ 10.01	22.21 $\pm$ 8.77	15.02 $\pm$ 10.27
Between	18.01 $\pm$ 5.60	11.89 $\pm$ 6.69	15.74 $\pm$ 5.45	8.59 $\pm$ 4.97
After LSTM	83.86 $\pm$ 0.74	83.70 $\pm$ 0.76	83.41 $\pm$ 0.89	83.38 $\pm$ 0.83

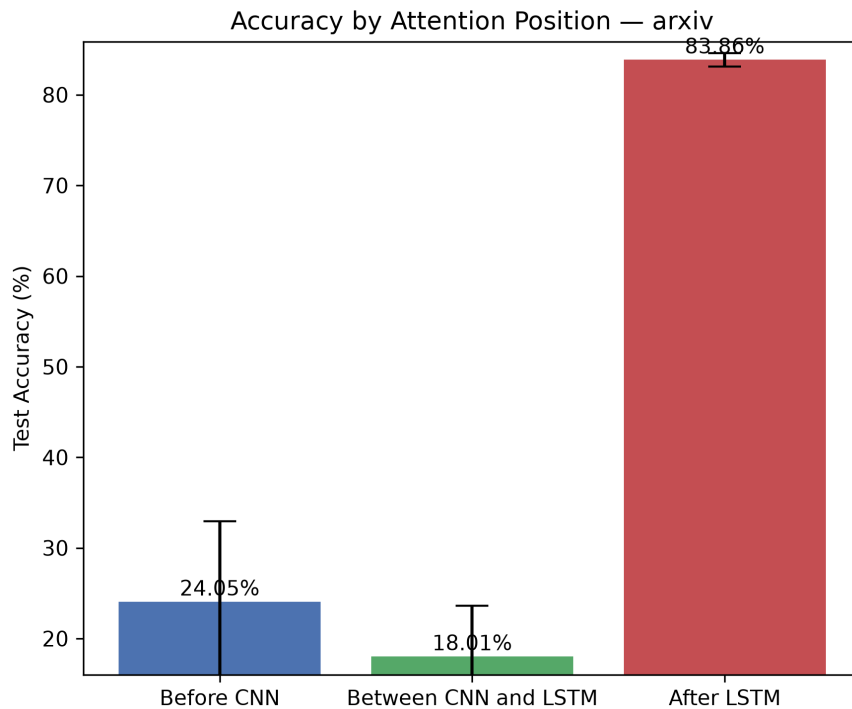


Figure 5: Test accuracy by attention placement on ArXiv, with error bars showing standard deviation across five seeds.

Results Discussion. ArXiv, the longest-sequence dataset in this study, shows the pattern from IMDB taken to its extreme. After-LSTM attention remains stable and accurate at 83.86%, close to its performance on IMDB despite sequences five times longer, and its F1-score of 83.38% confirms this is not driven by a small set of dominant classes. Before-CNN and Between, by contrast, collapse: 24.05% and 18.01% accuracy, respectively, both barely above the roughly 9% baseline for random guessing across eleven classes. Their F1-scores are even lower than their accuracy, 15.02% and 8.59%, which indicates that whatever these models did learn was concentrated on a

narrow subset of classes rather than spread across all eleven. Neither placement reliably learned this task under the standard three-epoch training budget used across all experiments.

Statistical Significance Analysis. After-LSTM significantly outperforms both earlier placements (Before-CNN: $t = -13.180$, $p = 0.0002$; Between: $t = -23.180$, $p < 0.0001$). Unlike on AG News and IMDB, the difference between Before-CNN and Between itself approaches significance ($t = 2.481$, $p = 0.068$), with Before-CNN scoring consistently higher, suggesting that on the longest sequences, even the relative ordering of the two weaker placements becomes more distinguishable. Section 4.4 places these three datasets side by side, and Section 5.3 investigates why Before-CNN and Between fail so sharply here specifically, using a follow-up convergence analysis.

4.4 Cross-Dataset Comparison

Figure 6 compares the three placements across datasets ordered by increasing sequence length. All three perform similarly on AG News, but their trajectories separate as the inputs grow longer. After-LSTM attention changes only modestly, from 91.32% on AG News to 84.93% on IMDB and 83.86% on ArXiv. In contrast, Before-CNN falls to 53.57% and then 24.05%, while Between falls to 65.18% and then 18.01%. The larger standard deviations for the earlier placements on IMDB and ArXiv also show that their degradation is accompanied by greater sensitivity to the random seed.

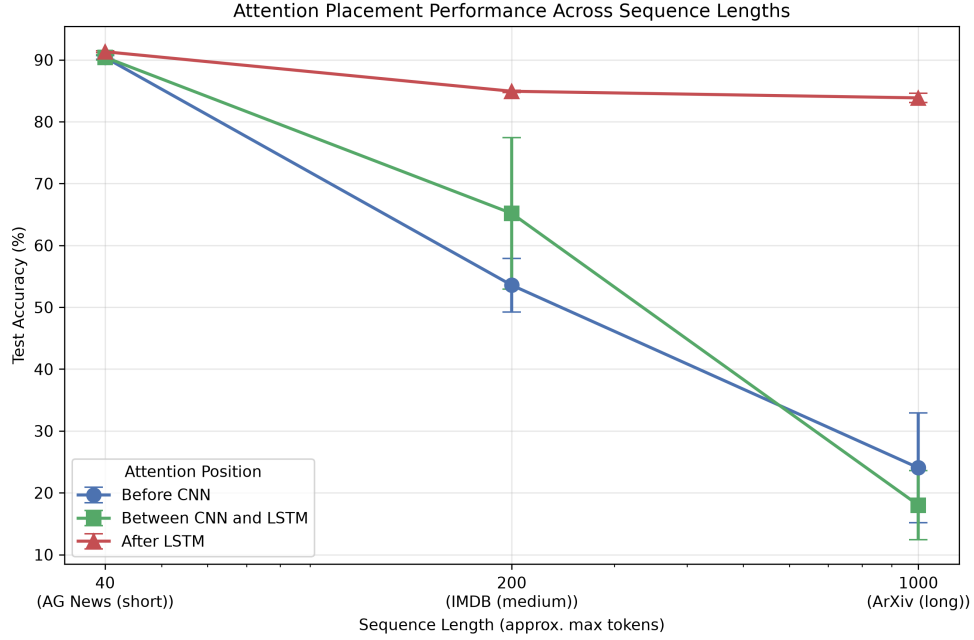


Figure 6: Test accuracy by attention placement across all three datasets, ordered by sequence length. After-LSTM attention remains stable while earlier placements degrade sharply as sequences grow longer. Error bars show the standard deviation across five seeds.

The widening separation is summarized directly in Figure 7 and Table 6. The advantage of After-LSTM attention is less than one percentage point on AG News, rises to 19.75–31.36 points on IMDB, and reaches 59.81–65.85 points on ArXiv. Thus, the effect of placement is not constant across tasks: it becomes much larger as the maximum sequence length increases from approximately 40 to 1,000 tokens.

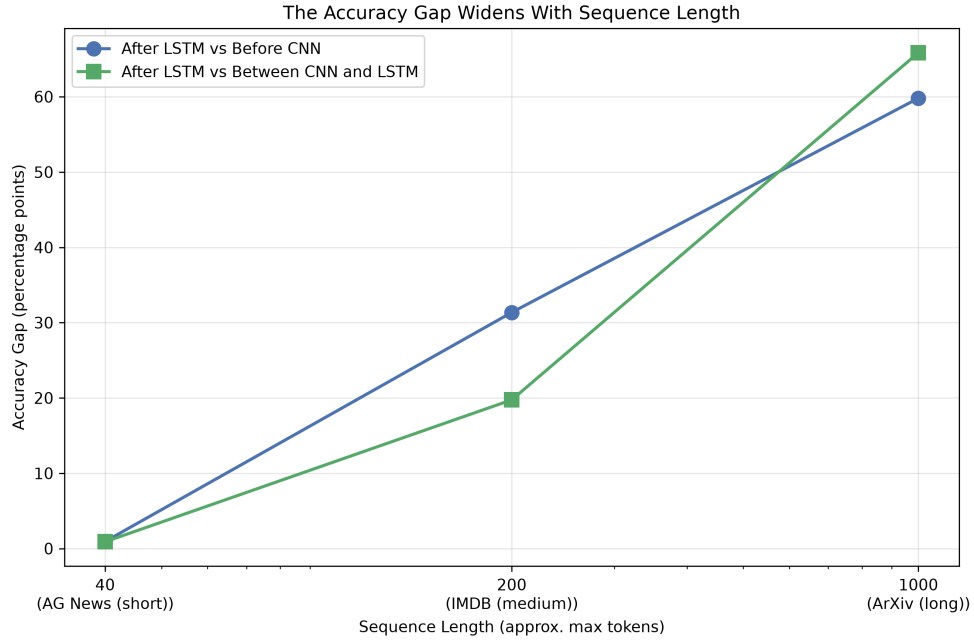


Figure 7: Accuracy gap between After-LSTM attention and each earlier placement as a function of sequence length.

Table 6: Accuracy gain of After-LSTM attention over earlier placements, by dataset (percentage points).

Comparison	AG News	IMDB	ArXiv
After LSTM vs. Before CNN	+0.94	+31.36	+59.81
After LSTM vs. Between	+0.90	+19.75	+65.85

Table 7: Statistical significance summary from paired t -tests across five matched seeds per dataset.

Dataset	Comparison	t	p
AG News	Before CNN vs. Between	−0.174	0.870
	Before CNN vs. After LSTM	−8.198	0.0012
	Between vs. After LSTM	−3.480	0.0253
IMDB	Before CNN vs. Between	−2.051	0.110
	Before CNN vs. After LSTM	−15.002	0.0001
	Between vs. After LSTM	−3.233	0.0319
ArXiv	Before CNN vs. Between	2.481	0.068
	Before CNN vs. After LSTM	−13.180	0.0002
	Between vs. After LSTM	−23.180	< 0.0001

Overall Discussion. Table 7 makes the cross-dataset pattern explicit. After-LSTM attention significantly outperforms both earlier placements on every dataset, and the magnitude of that advantage grows sharply with sequence length. Before-CNN and Between are statistically indistinguishable from each other on AG News and IMDB. On ArXiv, their difference approaches significance, with Before-CNN scoring consistently higher. Sequence length therefore does not merely add noise to earlier placements; it magnifies the performance and stability consequences of the placement decision. These results support treating attention placement as a sequence-length-dependent architectural choice rather than a fixed implementation detail.

5 Discussion

5.1 Effect of Sequence Length

The results across all three datasets point to one consistent pattern: attention placement matters, and how much it matters depends directly on sequence length. On AG News, with sequences truncated at 40 tokens, the three placements differ by at most one percentage point, a real but small effect. On IMDB, at 200 tokens, that gap widens to over 30 points, and on ArXiv, at 1,000 tokens, After-LSTM attention outperforms the earlier placements by roughly 60 to 66 points. This is not a fixed property of the architecture that happens to favor one placement; it is a property that emerges specifically as inputs get longer.

A plausible explanation lies in what each placement is actually doing. Attention placed before

the CNN or between the CNN and the LSTM reweights the sequence elementwise (Equation 3), leaving a full sequence for the CNN or LSTM to process further. Attention placed after the LSTM instead pools the entire sequence into a single vector (Equation 4). On short sequences, this distinction barely matters, since there is little for either operation to get wrong across only 40 tokens. On long sequences, an elementwise reweighting has to propagate its effect through many more subsequent computations before it reaches the loss function, while post-LSTM pooling sits directly before the classifier. This asymmetry provides a plausible explanation for why the performance of the earlier placements deteriorates more rapidly as sequence length increases.

The confusion matrix reinforces this picture. Figure 8 shows the aggregated confusion matrix for the Between placement on IMDB. Predictions are distributed fairly evenly across the two classes, but both rows contain substantial off-diagonal errors: 21,043 negative reviews are classified as positive, while 22,481 positive reviews are classified as negative. The model therefore does not simply collapse onto one dominant class; instead, it struggles to learn a reliable boundary between the two sentiment classes. This pattern is consistent with the placement’s mean accuracy of 65.18% and its high variability across seeds. By comparison, the 84.93% accuracy and low standard deviation of the After-LSTM placement indicate a substantially more reliable separation of the two classes.

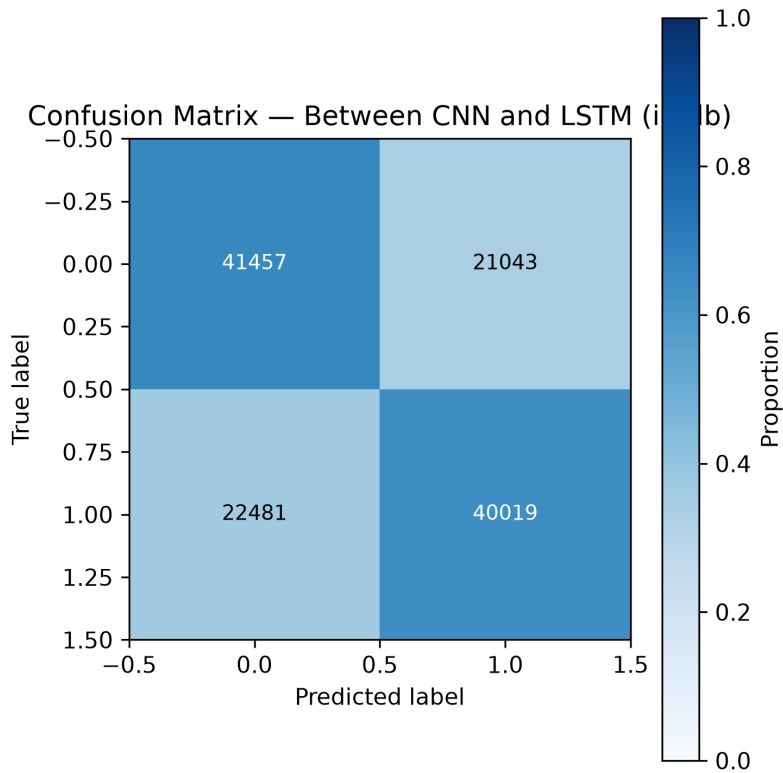


Figure 8: Confusion matrix for the Between placement on IMDB, aggregated across five seeds. Substantial off-diagonal counts for both classes are consistent with unreliable class separation.

5.2 Training Efficiency

Beyond accuracy, the three placements also differ in how much compute they use to get there, and here the picture is almost the reverse of what one might expect from their accuracy alone. Table 2 lists identical hyperparameters across all three variants, so the small differences in training time come from the placement within the architecture rather than from different training settings. On ArXiv, all three placements required approximately the same training time per run: 252 seconds for Before-CNN, 250 seconds for Between, and 251 seconds for After-LSTM. Training time itself therefore does not distinguish the placements meaningfully; all three process the same number of batches at a similar per-batch cost.

The real cost of poor placement is not additional wall-clock time per run, but the need for substantially more training to reach a comparable solution. Before-CNN and Between spend the same three epochs as After-LSTM, yet finish far from a useful solution on IMDB and ArXiv, as shown in Sections 4.2 and 4.3. In a practical setting, this means a practitioner using an early attention placement would need either to accept substantially lower accuracy or to train for more epochs in an attempt to close the gap, an additional cost that a correctly placed attention module does not incur under the standard training budget. Section 5.3 examines directly how much additional training is needed and whether it is sufficient.

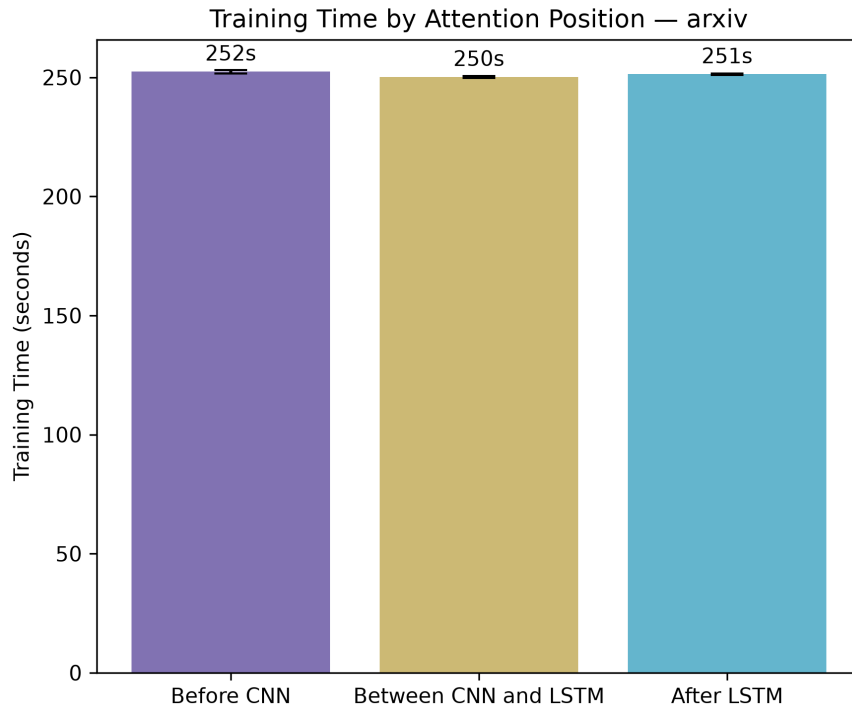


Figure 9: Average training time per run on ArXiv by attention placement. Differences in wall-clock time are small; the real cost of early placement is ineffective training rather than additional time per epoch.

5.3 Convergence Analysis

The results in Sections 4.2 and 4.3 leave open a question: are Before-CNN and Between fundamentally unable to learn on longer sequences, or are they simply too slow to converge within the three epochs used across all main experiments? To test this directly, we trained all three placements on ArXiv for ten epochs instead of three, using a single seed, and tracked accuracy and per-epoch training loss.

Table 8: ArXiv accuracy after three and ten epochs in the single-seed convergence diagnostic.

Position	3-Epoch Accuracy (%)	10-Epoch Accuracy (%)
Before CNN	17.4	72.4
Between	12.8	37.6
After LSTM	84.1	83.5

The pattern is informative. Before-CNN improved substantially with extended training, rising from 17.4% to 72.4%, confirming that it is capable of learning this task, just not within the standard training budget. Between showed a more unusual trajectory: its training loss remained close to the random-guessing baseline for the first five epochs, changing by less than 0.02, before dropping sharply from epoch six onward and reaching 37.6% accuracy by epoch ten. This delayed-breakthrough behavior suggests that the model spent several epochs in a poor region of the loss landscape before finding a useful gradient direction, rather than converging steadily throughout training. After-LSTM, by contrast, showed almost no change between three and ten epochs, moving from 84.1% to 83.5%, which indicates that it had already converged before the standard training budget was exhausted.

Taken together, these results indicate that early-placement instability is only partly a matter of insufficient training. Given enough epochs, Before-CNN and Between both improve considerably, suggesting that their poor initial performance is not caused by an inability to represent the task. However, neither placement closes the gap with After-LSTM even after more than three times the standard training, and Between’s stalled-then-sudden trajectory suggests that its optimization difficulty is qualitatively different from simply needing more time. Post-LSTM attention reaches a strong solution quickly and reliably; earlier placements reach a weaker solution more slowly and, in the case of Between, less predictably. Because this diagnostic uses only one seed, its exact trajectories should be interpreted as evidence about optimization behavior rather than as new estimates of average performance.

5.4 Threats to Validity

Several aspects of this study limit how far the findings should be generalized. First, the reweighting-versus-pooling distinction described in Section 3.3 means that the three placements are not mathematically identical operations moved to different depths; the operation itself changes depending on where it sits, since only the post-LSTM position can pool the sequence into a single vector without breaking the layers that follow it. This is an inherent property of inserting attention into a sequential architecture, not an implementation choice that could have been avoided, but it means the comparison isolates placement together with the operation it necessarily entails rather than placement alone in a stricter sense.

Second, all experiments use a single attention formulation, additive attention following Bahdanau et al. [2], and a single CNN–LSTM backbone with fixed layer sizes. Other attention mechanisms, such as multi-head or scaled dot-product attention, or different backbone configurations, might respond differently to placement, and this study does not test them.

Third, the convergence analysis in Section 5.3 uses a single random seed at ten epochs, chosen to keep the diagnostic experiment inexpensive relative to the 45 runs in the main study. The delayed-breakthrough behavior observed for Between is based on one training trajectory and may not represent every seed equally, although the three-epoch results across five seeds in Section 4.3 confirm that Between’s instability is consistent rather than a property of this single run.

Finally, all three datasets are text classification tasks. Whether the same length-dependent placement effect holds for other sequence tasks, such as sequence labeling or generation, remains untested.

5.5 Practical Implications

The results suggest a few concrete takeaways for anyone building a CNN–LSTM–attention model. For short sequences, attention placement is a minor design choice: all three positions perform within a percentage point of each other, and other considerations may reasonably take priority. For medium and long sequences, placement is not minor. The accuracy difference between the best and worst placements reaches over 60 percentage points on ArXiv, large enough to determine whether a model is usable at all. Post-LSTM attention was the most reliable choice across every dataset tested here, and given that all three placements cost roughly the same in training time, there is little practical reason to place attention earlier in the stack. More broadly, these results argue that attention placement deserves the same deliberate consideration as other architectural choices rather than being fixed by convention before a single experiment is run.

6 Conclusion

CNN–LSTM–attention models are common in applied sequence classification, but the attention layer is almost always placed at a single fixed point, chosen by convention rather than tested against alternatives. This paper studied that choice directly by comparing three placements—before the CNN, between the CNN and the LSTM, and after the LSTM—using one shared architecture, identical hyperparameters, and five random seeds per configuration across three datasets of increasing sequence length: AG News, IMDB, and ArXiv.

Attention placement affects performance, and the size of that effect grows with sequence length. On AG News, the three placements differ by about one percentage point. On IMDB, the gap widens beyond 30 points. On ArXiv, it exceeds 60 points. After-LSTM attention was the strongest and most consistent placement across every dataset tested, and paired-seed statistical testing confirmed that these differences are not explained by random variation. A follow-up convergence analysis showed that the earlier placements are not incapable of learning: extended training improved both substantially, but neither closed the gap with After-LSTM even after more than three times the standard training budget.

The placement of attention inside a CNN–LSTM architecture is therefore not a minor implementation detail. Its impact grows substantially with sequence length, and post-LSTM attention provides the most reliable performance across all datasets examined in this study. Future work could test other attention mechanisms, different CNN–LSTM configurations, sequence-labeling or generation tasks, and whether similar placement effects appear in Transformer-based hybrids.

References

- [1] Hochreiter, S., and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.
- [2] Bahdanau, D., Cho, K., and Bengio, Y. (2014). Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*.
- [3] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. In *Advances in Neural Information Processing Systems*.
- [4] Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H., and Bengio, Y. (2014). Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of EMNLP 2014*.

- [5] Alashjaee, A. M. (2025). Deep learning for network security: An attention-CNN-LSTM model for accurate intrusion detection. *Scientific Reports*, 15, 21856. DOI: 10.1038/s41598-025-07706-y.
- [6] Tuteja, G., Al-Yarimi, F. A. M., Ikram, A., Gupta, R., Ur Rehman, A., Singh, J., Delgado Noya, I., and Dzul Lopez, L. A. (2026). CNNAttLSTM: An attention-enhanced CNN-LSTM architecture for high-precision jackfruit leaf disease classification. *Frontiers in Plant Science*. DOI: 10.3389/fpls.2025.1720471.
- [7] Kuz, M., Kozlenko, M., et al. (2025). Research on a hybrid LSTM-CNN-attention model for text-based web content classification. *arXiv preprint arXiv:2512.18475*.
- [8] Sadhwani, S., Khan, M. A. H., Muthalagu, R., Pawar, P. M., and Suresh, K. (2026). A hybrid BiLSTM-CNN approach for intrusion detection for IoT applications. *Scientific Reports*, 16, 155. DOI: 10.1038/s41598-025-29079-y.
- [9] Islam, Md. S., et al. (2025). A novel deep learning framework with temporal attention convolutional networks for intrusion detection in IoT and IIoT networks. *Scientific Reports*, 15, 44624. DOI: 10.1038/s41598-025-32697-1.
- [10] Abdulganiyu, O. H., Fadi, O., Moukafih, Y., Ait Tchakoucht, T., Saheed, Y. K., Chukwuere, J. E., and Yau, S. (2026). Explainable attention based few shot LSTM for intrusion detection in imbalanced cyber physical system networks. *Scientific Reports*. DOI: 10.1038/s41598-026-38668-4.
- [11] Debnath, S., and Das, R. (2026). A hybrid CNN-LSTM intrusion detection framework for cybersecurity in smart renewable energy grids. *arXiv preprint arXiv:2606.25200*.
- [12] Dang, H., Liu, F., Stehouwer, J., Liu, X., and Jain, A. (2020). On the detection of digital face manipulation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 5781–5790.
- [13] Tsai, C. Y., Lin, C. M., Yang, C. H., Wang, Y. C., and Chiang, J. S. (2026). YOLO-AMC: An improved YOLO architecture with attention mechanisms for building crack detection. *arXiv preprint arXiv:2606.12958*.
- [14] Fraihi, N., Karrakchou, O., and Ghogho, M. (2025). Improving deep learning-based respiratory sound analysis with frequency selection and attention mechanism. *arXiv preprint arXiv:2507.20052*.

- [15] Mukherjee, S., Pramanik, P., Basuchowdhuri, P., and Bhattacharya, S. (2024). G4-Attention: Deep learning model with attention for predicting DNA G-quadruplexes. *arXiv preprint arXiv:2403.02765*.
- [16] Hallam, M. A., and Tseng, K. K. (2026). Fusion matters: Length-aware analysis of positional-encoding fusion in transformers. *arXiv preprint arXiv:2601.05807*.
- [17] Gokhman, R. (2026). LSTM variants for chaotic dynamical systems: An empirical study on the Lorenz attractor. *arXiv preprint arXiv:2606.22662*.
- [18] Shazeer, N. (2020). GLU variants improve transformer. *arXiv preprint arXiv:2002.05202*.
- [19] Mejhoudi, S. (2026). *Attention Placement in CNN–LSTM Architectures: Code and Reproduction Materials*. GitHub repository. Available at: <https://github.com/samihmejhoudi/attention-placement-project>